



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure
Environment
Actuators
sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
width  
height  
walls  
food_positions  
opponents  
agent_pos
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi Agent

The code explicitly instantiates and manages `self.opponents`. During step execution, these opponents independently select actions and move within the same environment. Their position changes directly affect the main agent's performance measure. Because the outcome depends on the actions of external entities, it is a multi-agent environment

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The complete history of everything the agent has perceived from its environment from the very beginning of its execution up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable

The agent only senses its current position, opponent locations, score, and whether it is currently touching food (smells_food) or a wall (hit_wall). But it cannot see the full layout of walls or food positions across the rest of the grid.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Evaluating performance based on external environment results rather than internal processing effort ensures that the agent is judged by what it actually achieves in the real world, not by how hard it tries.

If performance were measured by internal effort, an agent could maximise its score by simply running endless complex loops without ever moving or successfully navigating the grid. Setting metrics based on objective environment outcomes prevents metric exploitation and forces the agent to learn useful, rational behaviour.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate

tuples safely avoiding position $(0, 0)$, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?**

Adding `self.toxic_traps` while hiding them from sensors introduces unobserved state components, making the environment more severely partially observable. The agent cannot perceive or predict trap locations in advance, creating hidden environmental hazards that affect its performance without prior sensory feedback.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. Implementation Instructions: Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

By providing `'smells_toxin'`, the agent receives immediate feedback when occupying a hazardous tile. This sensory data allows a rational agent to update its internal belief state to mark that location as dangerous, adjusting its policy/action selection to avoid returning to that cell and thus maximizing expected utility by avoiding repeated score penalties.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. Implementation Instructions: In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example

discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World Exploit occurred when an agent was rewarded solely for cleaning dirt. To maximize its score, the agent repeatedly cleaned a square, dumped dirt back out, and cleaned it again endlessly. Adding score penalties for traps prevents similar metric exploitation by ensuring the agent prioritises survival and goal completion over irrational behaviours.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING